
**Mansoura University**
**Faculty of Computers and Information**
**Department of Computer Science**


**Spring 2026**

# [SWE143] Software Project Management

## Lecture 04: Software Development Activities – Part 2

Level 300:software engineering program

**Muhammad Haggag Zayyan, Ph.D**

## ❑ Software development phases:

- Requirements and User Experience Design
- Design
- Implementation
- Verification and Validation
- Deployment
- Operations and Maintenance

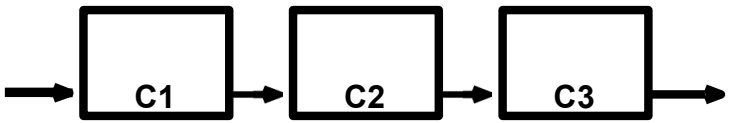
# System Design

- Goal:
  - Defining the structure of the software to build (= system architecture)
- Outputs (diagrams/text):
  - components which constitute the system
  - functions each component implements (role)
  - how the components are interconnected
- The activity is relevant also for managerial reasons: the system architecture provides a "natural" decomposition of work
- The definition of a system architecture can be based on a pattern or predefined blueprints (high level structure):

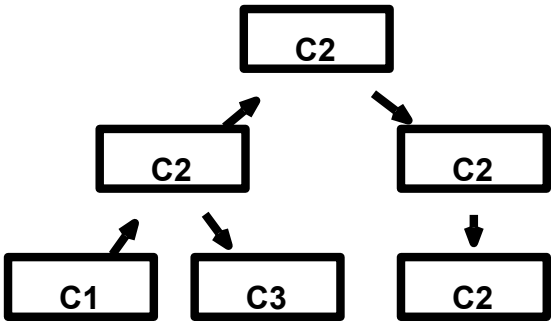
➢ **Monolithic Architecture**: A single codebase containing all components and functions.

➢ **Microservices Architecture**: Separate services responsible for individual functions, each with its own lifecycle.

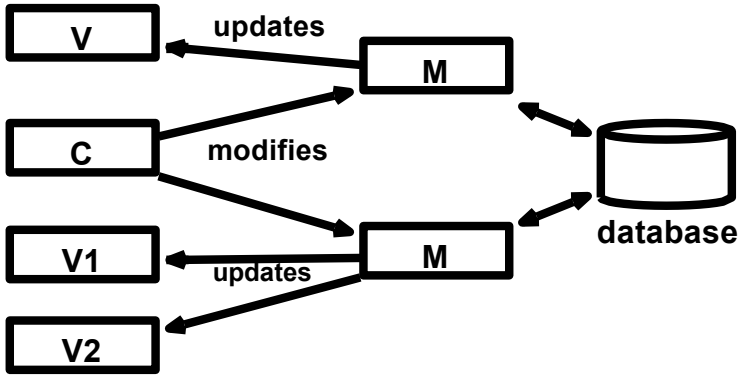
# Architectural Patterns



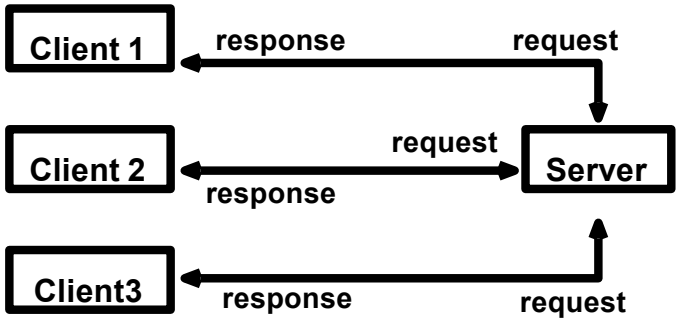

A Pipeline Architecture




A Layered Architecture



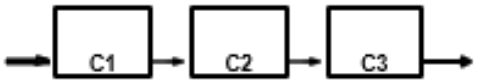

A Data-Centric applicaton with two MVCs




A client-server Architecture

# Architectural Patterns

- Pipe and filter
  - Composition of
    - a chain of data processing units (filter) which perform transformations.
    - Filters such as: Parsing, Transformation, Aggregation, Validation, Sorting
    - and pipe, which serve as connectors for the stream of data being transformed, each connected to the next component in the pipeline
  - The processing is usually sequential, but filters can be implemented in parallel if needed (this depends on the specific design or use case) such as:
    - Filter 1 transforms the text to uppercase.
    - Filter 2 removes punctuation.
    - Filter 3 counts the number of words in the text.

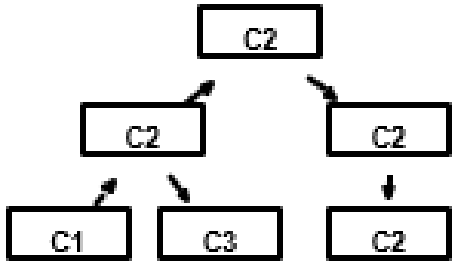



A Pipeline Architecture

  - Focus: I/O specification
    - defining how the **input** and **output** data are handled by each **filter** and the **pipe**

# Architectural Patterns

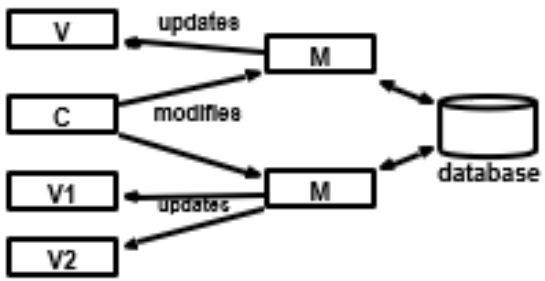
- Layered/Hierarchical
  - –Hierarchy of components - organized into multiple control levels
    - –Higher levels often managing more complex functions
    - –Lower levels of the architecture perform simpler functions
  - –Focus: control and information flow;
    - –block responsibilities: it refers to how responsibilities are assigned to different layers
    - –Example: in embedded systems, sensor-actuator
      - At the lower level, **sensors** are responsible for reading data from the environment [perform simpler functions]
      - At the higher level, a **controller** takes the input of the sensors and decides the action to perform




A Layered Architecture

# Architectural Patterns

- Data-Centric
  - When data storage is central: Model View Controller (MVC) pattern
    - The model defines how data are to be stored and manipulated. (defining data schema: tables. Operation: sql)
    - The view defines how data are to be presented to the user. (UI)
    - The controller defines the logic of the operations (processing user input, making decisions, and updating the model accordingly.)
  - Focus: data model, operations
  - Many web applications and many desktop applications use the data-centric architectural style

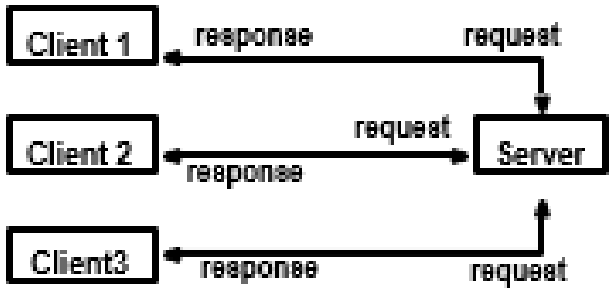



A Data-Centric applicaton with two MVCs

In the MVC architecture, developing different view components for your model component is easily achievable. It empowers you to develop different view components, thus limiting code duplication as it separates data and business logic

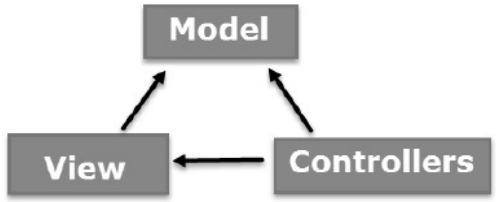
# Architectural Patterns

- Client-server
  - Split functions: Server (main functions) and clients (requesting services)
  - Focus: communication protocolo/service specifications




A client-server Architecture

# ASP.NET MVC Pattern




- MVC is a design pattern used to decouple user-interface (view), data (model), and application logic (controller). This pattern helps to achieve separation of concerns.
- Using the MVC pattern for websites, requests are routed to a Controller that is responsible for working with the Model to perform actions and/or retrieve data. The Controller chooses the View to display and provides it with the Model. The View renders the final page, based on the data in the Model.

**Models & data**
Create clean model classes and easily bind them to your database. Declaratively define validation rules, using C# attributes, which are applied on the client and server

**Controllers**
Simply route requests to controller actions, implemented as normal C# methods. Data from the request path, query string, and request body are automatically bound to method parameters.

**Views with Razor**
The Razor (view engine) syntax provides a simple, clean, and lightweight way to render HTML content based on your view. Razor lets you render a page using C#, producing fully HTML5 compliant web pages

```
public class Person
{
    public int PersonId { get; set; }

    [Required]
    [MinLength(2)]
    public string Name { get; set; }
```

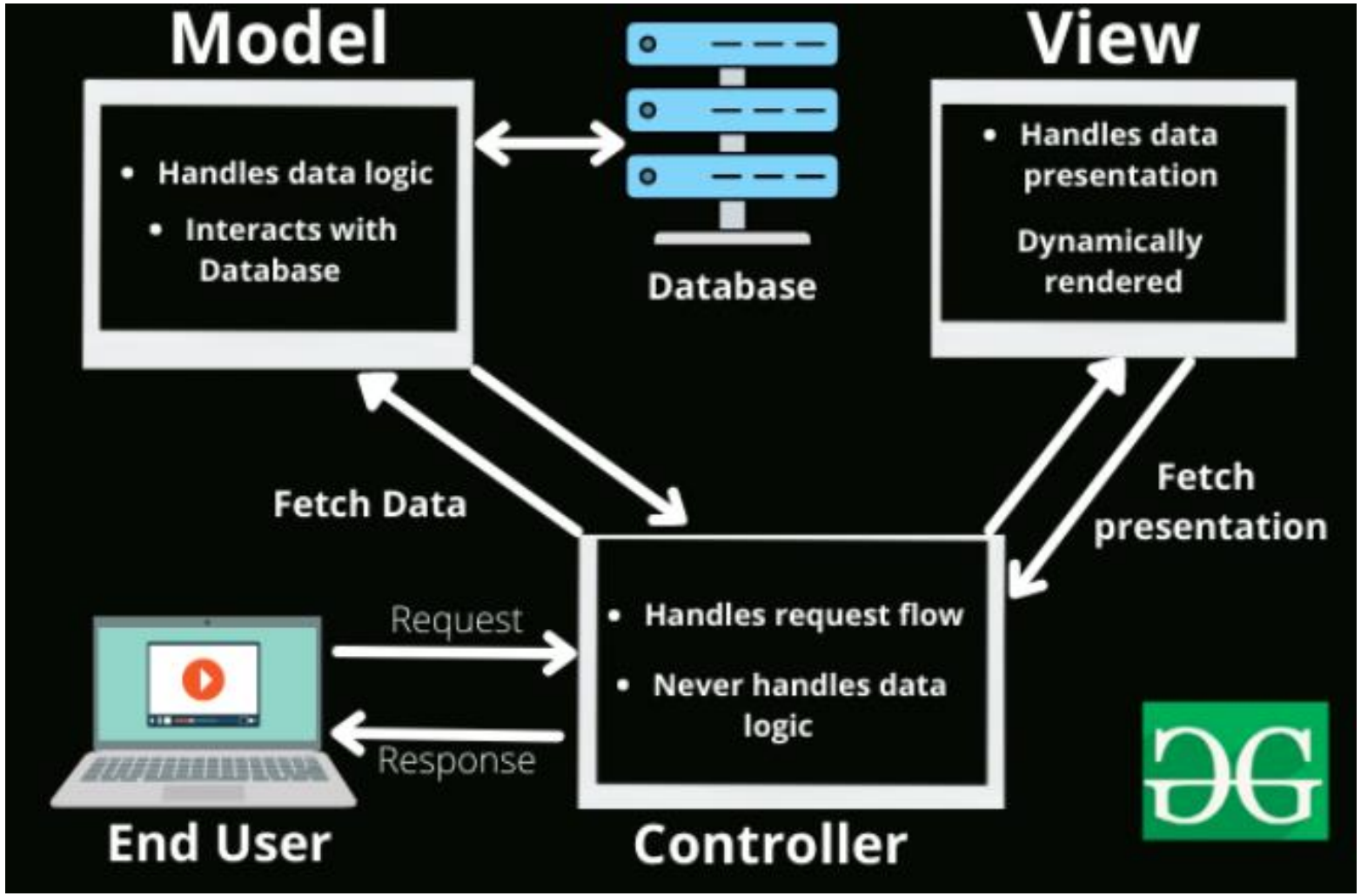
```
// GET: /people/details/5
public async Task Details(int id)
{
    var person = await _context.People.Find(id);

    if (person == null)
    {
        return NotFound();
    }

    return View(person);
}
```

```
<table class="table">
    <thead>
        <tr>
            <th>@Html.DisplayNameFor(model => model.Name)</th>
            <th>@Html.DisplayNameFor(model => model.PhoneNumber)</th>
            <th>@Html.DisplayNameFor(model => model.Email)</th>
        </tr>
    </thead>
    <tbody>
```

# Components of MVC



# System Architecture Views and Their Corresponding UML Diagrams

- **The logical view** identifies the main **elements** and data structures of the system to build. It is best described with class and sequence diagrams.
- **The component view** provides a programmer-oriented view of the system. It is mainly concerned with the **components** to be developed and is best described in UML with class and component and package (namespaces) diagrams.
- **The process view** provides a specification of the **behavior** of the system: interactions among components and the sequence of actions that are required to implement the user functions. It is best described with sequence and communication diagrams.
- **The physical view** provides a specification of the physical **deployment** of a system, that is, on what computer or process each element of the architecture will run. It is best described by a deployment diagram.
- **The use case view** describe the **high-level functions** and scope of a system. These diagrams also identify the **interactions** between the system and its actors. The use cases and actors in use-case diagrams describe what the system does and how the actors use it, but not how the system operates internally.

- Software development phases:
  - Requirements and User Experience Design
  - Design
  - Implementation
  - Verification and Validation
  - Deployment
  - Operations and Maintenance

# Implementation

- Goal:
  - Writing the code!
- Some of the Project Management-relevant activities during implementation:
  - Collection of productivity and size metrics
    - Measure speed/size (LOC) of delivered code [assess the amount of code delivered over time]
    - Productivity tracking helps assess the progress of individual developers or the entire team. While LOC is one measure, other factors such as code complexity and **completed features** are equally important.
  - Collection of quality metrics
    - Code coverage: The percentage of code tested by automated tests.
  - Use of coding and documentation standards
    - describing best practices – work of different programmers is similarly structured
    - Code management practices (versioning; code releasing standards)

# Software Quality Metrics

- **Number of lines, files.** How do your file sizes affect your software? What is the function of the code lines?
- **Field bugs.** What are the problems in the running software? How many bugs did you find in production? Is your software reliable?
- **Code churn.** (also interchangeably known as rework) Why is one part of the code churning more than others? Why is it error-prone? Is anything in the completion rate standing out?
- **Static analysis findings.** How long does it take to fix code? Is the software **secure** [include isolating and securing the database, encoding data, data validation]?
- **Bug arrival rate.** What are the **rates** at which bugs are coming in? How many software releases happened during a period?
- **Performance.** Does the software code last during updates? Is it performing as it should? Do users enjoy it?
- **Test failures.** Automated and manually, what tests are failing? What is your failure balance?

- Software development phases:
  - Requirements and User Experience Design
  - Design
  - Implementation
  - Verification and Validation
  - Deployment
  - Operations and Maintenance

# Verification and Validation

- Verification = did we build the system right?
  - ❑ Set of activities to ensure the system implement the requirements correctly.
- Validation = are we building the **right** system?
- Collectively known with the acronym **V&V**
- Part of quality management
- The main (but not the only) way of performing V&V for software systems is testing - Other methodologies:
  - **Formal verification** methods rely on mathematically rigorous procedures to search through possible execution paths of your model or code to identify errors in your design.
  - **Code inspection** is a type of Static testing that aims to review the software code and examine for any errors. It helps reduce the ratio of defect multiplication and avoids later-stage error detection

# Types of Testing

- Unit testing
  - Scope: a piece of code, such as a class – automated testing
- Integration testing
  - Scope: the interaction between two components - looks for inconsistencies (array->string)
  - Mars Climate Orbiter bug: two components used different units (metric and imperial); ~400M USD loss.
- System testing
  - Scope: the system behaves as expected and implements correctly all the requirements
  - Test cases - verify whether a system implements the requirements
- Usability testing
  - running in parallel with the other testing activities
  - Scope: verifying whether the user experience and interaction is intuitive, effective, and satisfying
  - Used to reduce the probability of human errors (safety-critical systems).

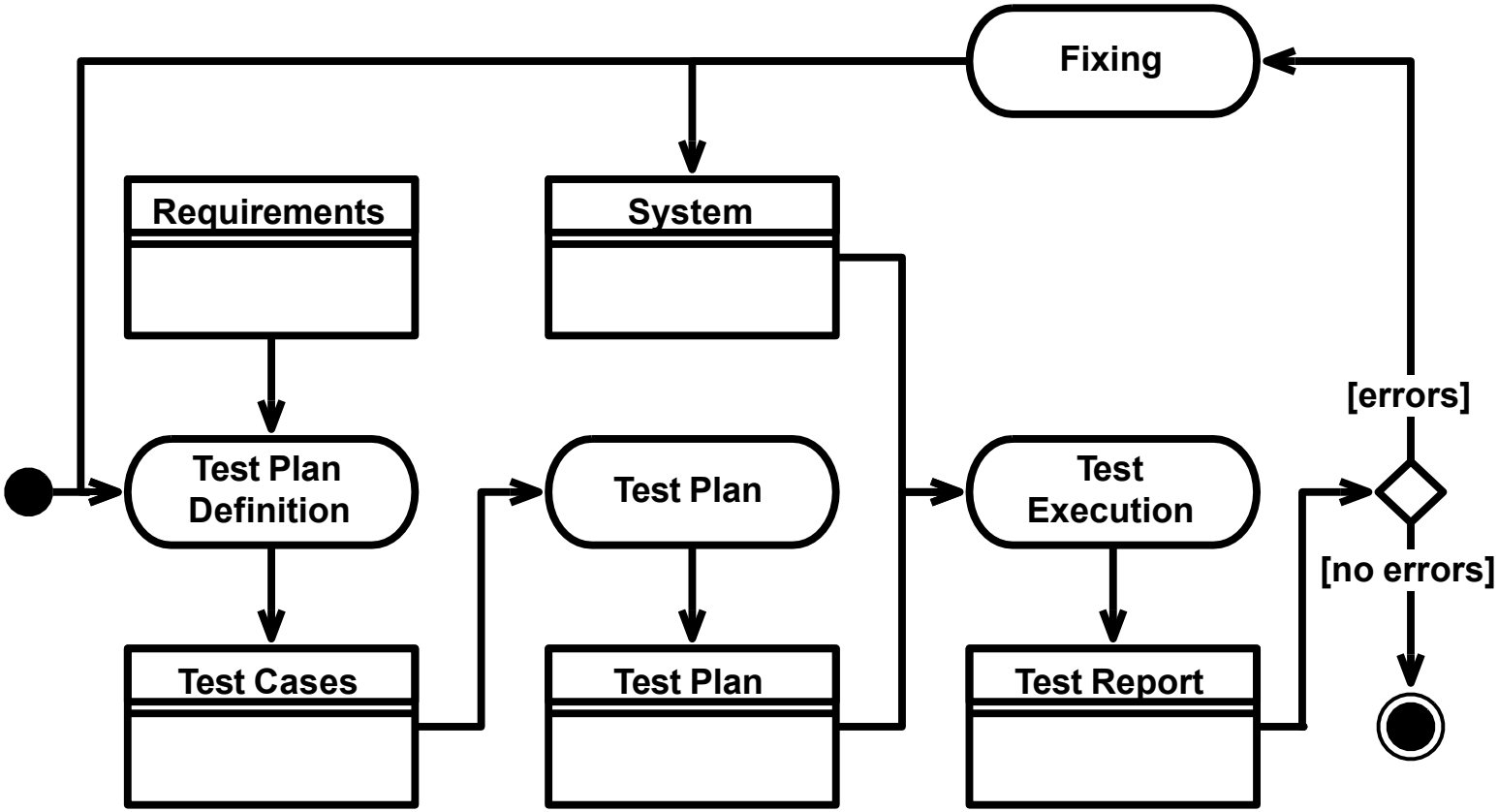
# Organizing Testing Activities

Unit tests are organized in the following two steps:

- **Test plan definition**
  - Goal: Listing a set of test cases
  - Different test cases need to be defined for each requirement.
  - Verifies either a particular condition specified by the requirement in different operational conditions
  - Traceability information, which links a test case to the requirement it tests, helps manage changes and the overall maintenance process

- **Test execution and reporting**
  - Goal: the team or automated procedures execute tests and report on the outputs, that is, which tests succeeded, and which failed.
  - Manual test execution is time consuming and demands commitment due to:
    - Testing is the last activity before releasing a system. It requires quite some commitment to work hard to demonstrate that system in not working.
    - When an error is found, the process stops till a fix is found. When the fix is ready, it is necessary to start the testing activities all over again.
      - This is to ensure that there are no regressions, that is, working functions have not been unintentionally broken by the fix.

# The System Testing Process

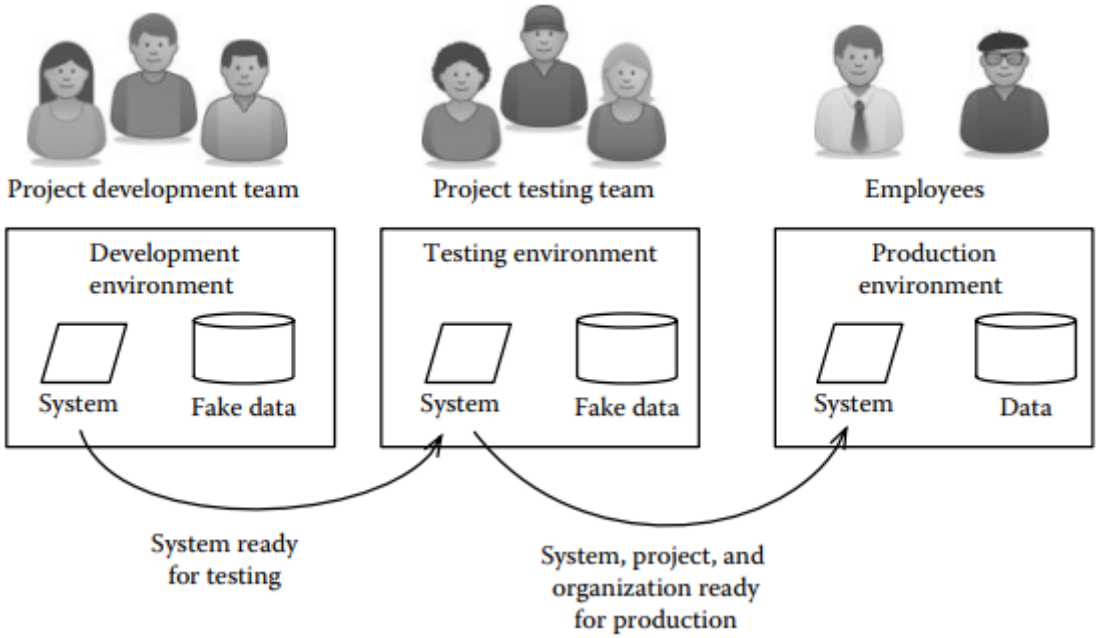


- Software development phases:
  - Requirements and User Experience Design
  - Design
  - Implementation
  - Verification and Validation
  - Deployment
  - Operations and Maintenance

# Deployment

- Goal
  - Installing the new system and making it operational
- Some concerns:
  - Ensuring continuity of business operations (replace old system)
  - Migrating data
  - Transitioning to operations and maintenance
- Factors to consider to ensure smooth transition of operations when the new system is ready
  - The human factor: is the people ready to use the system?
  - The data factor: is all the data which is needed for the system to run available to the new software?
  - The hardware factor: are all interfaces ready and functional?

# Development environments




The new system should cause any interruptions by ensuring that the software evolution must not interfere with production.
Achieved by: Providing three exact and independent replicas of the same operating environment

- **A development environment**, where the actual development of the software takes place. The development environment is completely isolated from production,
- **A testing environment**, where the team tests a system that is ready for deployment. The testing environment is isolated from the development and the production environment, so that, on the one hand, no changes made by developers interfere with testing and vice versa.
- **A production environment**, where the system is actually used. Any change to the production environment interferes (positively or negatively) with the operations for which a system is use

# Migration strategy approaches

- Migration strategy, defines an approach to the introduction of the new system.
  - Cut-over: the new system completely replaces the old one.
  - Parallel Approach: the old and the new system operate simultaneously for a period –new system can be tested before takes place. (Both systems run concurrently)
  - Piloting: the new system is installed for a limited number of users or for a specific business unit – both old and new systems are alive.
  - Phased Approach: functions are rolled out incrementally.

- Cut-over → all at once
- Parallel → both systems run together
- Piloting → test with a small group
- Phased → introduce step by step

# Release process

When the strategy is agreed upon, the final step is the implementation of the release process, which in turn consists of the following steps:

- **Deliver training**, to ensure that the users acquire the necessary skills to use the new system.
- **Perform data migration**, which includes updating the data used in the production environment so that it can be used by the new system.
- **Install the new system**, which puts the new system in production.
- **Set up the support infrastructure**, namely, set up an infrastructure to support operations - Bug tracking system.

- Software development phases:
  - Requirements and User Experience Design
  - Design
  - Implementation
  - Verification and Validation
  - Deployment
  - Operations and Maintenance

# Operations and Maintenance

- Goal
  - Ensuring the system runs smoothly after its release
- Activities:
  - Providing Technical Support
  - Monitoring system performance
  - Trigger maintenance activities
  - Collecting and managing tickets (clarifications, bugs, requests for improvement)

# Supporting and Monitoring Operations

- Operations are outside the scope of a project.
- Support activity is planned after a system is released to ensure that the project outputs meet the quality goals and the transition to operations is as smooth as possible – **achieved by:**

  - **Providing technical support**. The support is meant to help users get aware of the system and it can be organized as a help-desk collecting tickets from users.
    - Some of these tickets are requests for clarifications on the use of the system. Others will signal malfunctions, glitches, and requests for improvement.

  - **System monitoring.** A set of metrics might be collected on the system after its initial release to monitor performances, issues, and other system features.

# Types of Maintenance

- Corrective, تصحيحية if relative تتعلق بto fixing an issue discovered after the release of the system. E.g., a bug in the application that causes the system to crash
- Preventive,وقائية if relative to fixing an issue discovered, but not occurred (or, at least, signaled by users) [proactivelyاستباقية] (planned maintenance). E.g., update system security
- Adaptive,تكيفية if relative to adapt a system to changed external conditions – in case of OS upgrade. E.g., re-compile code to run on new OS.
- Perfective,تحسينية if relative to improve some characteristics of a system. E.g., stakeholders request faster response time or better user interface design.

Perfective and corrective maintenance are triggered by suggestions and **bug reports** sent by users. Suggestions and bug reports are also called **issues** or **tickets**.

# Summary of types of maintenance:

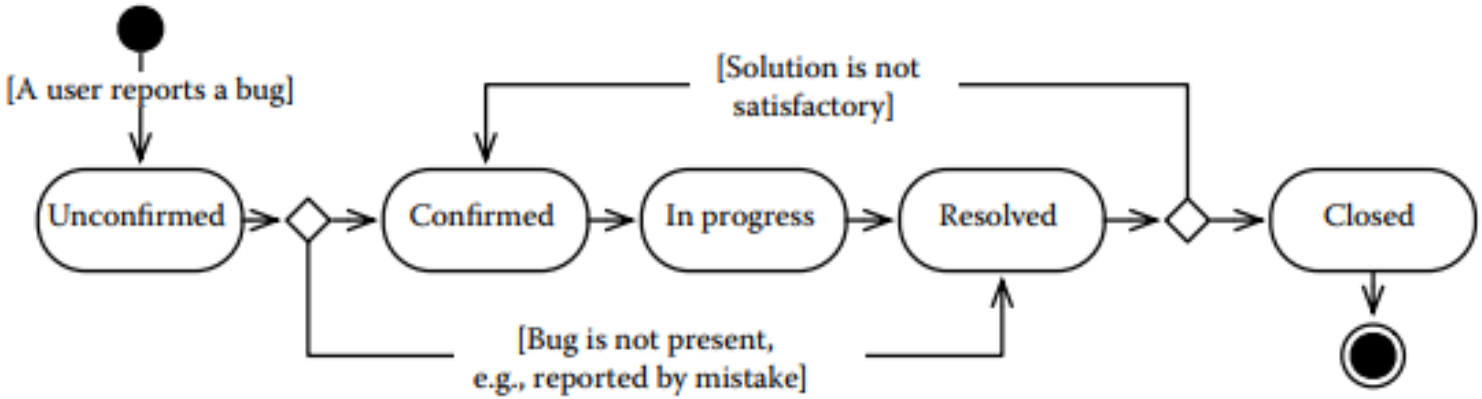
| Type of Maintenance | Definition | When Applied | Triggered By | Examples |
|---|---|---|---|---|
| **Corrective Maintenance** | Fixing issues or bugs that occur in the system after release. | When bugs or issues affect functionality. | Bug reports, error logs, or system monitoring. | Fixing crashes, resolving functionality issues. |
| **Preventive Maintenance** | Proactive updates to avoid potential future issues or risks. | To prevent problems that haven't yet occurred. | Risk analysis or anticipated system changes. | Updating security, optimizing code to avoid future problems. |
| **Adaptive Maintenance** | Updating the system to adapt to external changes (OS updates, hardware changes, etc.). | When external conditions or requirements change. | External changes (e.g., OS upgrades, third-party changes). | Adapting to new OS, integrating new hardware. |
| **Perfective Maintenance** | Enhancing system performance or adding features based on user feedback and evolving requirements. | When improvements are desired to enhance performance or user satisfaction. | User feedback, performance issues, feature requests. | Improving performance, adding new features, UI redesign. |

# Organizing Support and Maintenance Activities

Goal: keeping formal track of the tickets can be achieved by:

- **Defining a workflow for tickets**, which describes how bug reports are formally tracked and managed.

- **Automating the collection and management of tickets**. This is usually achieved by introducing a bug tracking system.
  - A bug tracking system allows one to maintain a list of tickets and trace their workflow states.




The lifecycle of a bug

# Thank You